

FINAL REPORT

Plant Disease Recognition from Leaf Images

Dataset B: PlantVillage — Plant-Disease Classification

Plant Recognition · Capstone Project · Step 4

This report brings Deliverable 1 (exploration, visualisation and pre-processing) and Deliverable 2 (modelling, selection and evaluation) together into one account of the project, and closes with the conclusions and outlook that neither could draw on its own. It runs from 54,305 raw leaf photographs to a fine-tuned DenseNet-121, evaluated once at the very end against a partition of the data kept sealed throughout development.

Cohort may26bds_int

Prepared by the project team

Adrian Bogdan RUS · Alexander DANZ · Christoph Clemens Martin NEDDENS · Scheima Sara OBEIDI

Project mentor: Habiba El Husseiny (ML Engineer)

6 August 2026

Contents

1. The problem, and why it is hard..... 3

2. The data, and what we found in it..... 3

3. How the four difficulties were answered..... 6

4. What "good" means here..... 8

5. The ladder of models..... 9

6. Choosing the final model..... 12

7. The final model, and the sealed test set..... 13

8. Where the remaining errors are..... 14

9. Looking inside the model..... 15

10. Difficulties encountered along the way..... 17

11. What this report does not establish..... 17

12. Outlook..... 18

13. Conclusion..... 19

Appendix A. The eleven models compared..... 20

1. The problem, and why it is hard

A grower who sees something wrong with a crop has a short time in which to act, and what to do depends on knowing what the problem is. That knowledge normally comes from an expert inspection, which is slow, expensive, and least available in the places where crop loss does the most harm. A phone camera and a model that can read a leaf would close that gap: photograph a leaf, get a likely diagnosis, act earlier, lose less of the crop. That is the idea behind applications such as PlantNet, and it is the idea behind this project.

Plant Recognition builds two separate image classifiers. One identifies a plant's species. The other identifies its disease. This report is about the second one. The goal was set at the start and has not changed since: identify the crop and its condition from a single photograph of one leaf, accurately enough to be worth showing to a grower, and honestly enough that we know where the accuracy stops.

Why the task is harder than it looks

The model is not being asked to tell a leaf from a lamppost. It is being asked to tell Tomato Early blight from Tomato Late blight. These are two diseases of the same crop. They appear on tissue that is otherwise identical, and what separates them may be a few square centimetres of texture. In image recognition this is called *fine-grained classification*, and it sits at the harder end of the field.

There is a second difficulty hidden in the label set. Twelve of the 38 classes are healthy tissue. So the model has to make two very different kinds of judgement at the same time: telling a diseased leaf from a healthy one, which is usually easy, and telling two diseases of the same crop apart, which is often hard. The classes are not equally difficult, and any single average across them will hide that.

Almost everything in this report follows from those two facts — the choice of model, the choice of measurement, the shape of the errors that survive at the end, and the one thing the finished model still cannot be trusted to do.

Three names used throughout

Three models come up often enough to be worth naming now, so that later sections read more easily.

- **The baseline** is a Random Forest built on roughly 160 hand-designed features. It is the best result we achieved without deep learning, and it is the mark the neural networks had to beat.
- **The scratch CNN** is a convolutional network trained from a random start, with no borrowed knowledge.
- **The final model** is DenseNet-121, the model we chose in the end.

How this report is organised

The report follows the order in which the work was actually done, because that order is also the argument. Section 2 covers what the data turned out to be, and ends with the four difficulties the exploration uncovered. Section 3 answers those four difficulties one at a time. Section 4 explains how we decided what counts as a good result. Section 5 walks up the eleven models, each of which was built to answer one specific question. Sections 6 to 9 cover the model we chose, how it performed on data no model had seen, where its remaining mistakes are, and what can be seen inside it. Sections 10 to 13 set out what went wrong along the way, what the work does not prove, what should happen next, and what it all adds up to.

2. The data, and what we found in it

Where the data came from

The dataset is PlantVillage, downloaded from Kaggle in its original, unmodified form. It is freely and publicly available under its Kaggle licence. We deliberately used the raw images rather than a version that had already been split or augmented by someone else. That way every change made to the data afterwards is one this project chose, can explain, and can repeat.

Each image is labelled by the folder it sits in, using a two-part name of the form *crop__condition* — for example Tomato__Late_blight or Apple__healthy. Reading those folder names gives **14 crops** and **21 conditions**

("healthy" plus twenty named diseases), which combine into **38 classes**. It is worth being precise here, because it is easy to describe wrongly: the 38 classes are crop-and-condition pairs, not 38 diseases. Twelve of them are healthy tissue. Tomato alone accounts for ten of the classes, Apple and Corn for four each.

The download contains three parallel versions of every photograph: colour, greyscale, and segmented with the background removed. We checked that all three hold the same 38 classes with matching image counts, which confirmed they are the same pictures in different dress rather than extra data. We therefore worked with the colour version: greyscale would throw away colour that a disease model may need, and the segmented version was kept in reserve as a possible defence against background bias. Section 9 describes what we eventually used it for. Restricted to colour, the working set is **54,305 images**, each a uniform 256×256 -pixel, three-channel square.

How we looked at 54,305 images

Images cannot be examined with ordinary table tools, so the first step was to build a table about the images. Each row is one image, and the columns record things we can measure: width, height, pixel count, number of colour channels, file size, overall brightness, and the average of the red, green and blue channels. Alongside those sit the label and the crop, condition and healthy-or-diseased status read out of it.

We used two passes for this, for a practical reason. The exact class counts come from counting every single file, which is fast because it never opens an image. The pixel measurements come from a random sample of up to 250 images per class — 9,402 images in total — cached to disk so the slow scan only ever runs once. One class, healthy Potato, has only 152 images to draw from. Taking the same number from every class keeps even the rarest classes properly represented in the sample.

That table was then summarised into a fifteen-row data audit, one row per variable, recording its type, how much of it is missing, whether it is categorical or numeric, a short description of its distribution, and a note on whether it matters for modelling. It is reproduced below without the "type" and "missing" columns, because every variable is stored in the obvious type and **not one of the fifteen has a single missing value**.

Variable	What it records	Distribution	Why it matters
filepath	Path to the image; unique identifier	9,402 sampled paths	Not predictive; used to load the image
label	Full class crop___condition — the 38-class target	38 classes; 5,507 (max) to 152 (min)	Extreme imbalance ~36:1. This is what we predict
crop	Plant type parsed from the label	14 crops; Tomato is the largest	Entangled with disease; several crops single-condition
condition	Specific condition or disease parsed from the label	21 distinct conditions	The fine-grained disease label
is_healthy	True when the condition is exactly "healthy"	31% healthy in the sample	Core healthy/diseased axis; only meaningful for crops with both
width	Image width in pixels	constant 256 px	Uniform, so no resizing needed for consistency
height	Image height in pixels	constant 256 px	Uniform; images are 256×256
aspect_ratio	Width divided by height	constant 1	Square, so no distortion from a square resize
n_pixels	Width $\times$ height (resolution)	constant 65,536	Uniform resolution
channels	Number of colour channels	constant 3 (RGB)	Already standardised to three channels
file_size_kb	File size on disk	min 5, median 16, max 30 KB	On fixed-size JPEGs, larger $\approx$ more detail; weak signal
brightness	Mean pixel intensity (0–255)	mean 118 of 255	Controlled lab lighting; fairly uniform
mean_R	Mean of the red channel	mean 120	Colour highly redundant with brightness

Variable	What it records	Distribution	Why it matters
mean_G	Mean of the green channel	mean 126	Dominant channel (foliage)
mean_B	Mean of the blue channel	mean 108	Lowest channel

Table 1. The data audit. Class counts come from the full file census; pixel-feature distributions from the 250-per-class sample. The audit is also exported as DATA_AUDIT_plantvillage.xlsx.

Reading the audit, three things stand out.

- **Nothing is missing.** Not one variable has a single missing value, so no gap-filling is needed anywhere.
- **The images are already uniform.** Width, height, aspect ratio, pixel count and channel count are all constant — every image is a 256×256 three-channel square. No resizing is needed to make them consistent.
- **The target is very unevenly distributed**, at roughly 36 to 1, and the colour variables are visibly redundant — green is the dominant channel and all three track overall brightness. Both of these are confirmed properly below.

After that we produced five figures. Following the project methodology, each was read twice: once for what it means for the project in plain terms, and once as a statistical test to check that the visual impression is real and not wishful thinking. The tests used were chi-square for the category questions, a one-way ANOVA with an effect size for the colour comparison, and Pearson correlation for feature redundancy.

Finding 1 — the classes are very unevenly sized

Sorting the classes by size makes the imbalance obvious at a glance. The largest class, Orange citrus-greening, holds **5,507 images**. The smallest, healthy Potato, holds **152**. A few large classes tower over a long tail of small ones.

A chi-square goodness-of-fit test against an equal-size expectation gives $\chi^2 \approx 41,874$ with $p \approx 0$. That sounds dramatic but says very little on its own: with 54,305 images the test was almost certain to reject. The number that actually measures how bad the problem is, is the plain ratio — **36.2 to 1**. The practical consequence is that overall accuracy will flatter any model we build, and the imbalance has to be corrected during training.

Finding 2 — crop and disease are tangled together

Splitting each crop into its healthy and diseased images shows that the balance varies a great deal from crop to crop. Tomato dominates, at roughly a third of all images, and is overwhelmingly diseased. More importantly, **five crops appear under one condition only**: Blueberry, Raspberry and Soybean are entirely healthy, while Orange and Squash are entirely diseased.

A chi-square test of independence rejects the idea that crop and disease status are unrelated ($\chi^2 \approx 28,005$, 13 degrees of freedom, $p \approx 0$, with the smallest expected count around 103, comfortably above the usual threshold of five). Crop and disease status are therefore entangled. The single-condition crops are the decisive detail: for those five crops, a question of the form "is this plant healthy or diseased?" cannot be learned at all, because the data contains no counter-example.

Finding 3 — a leaf's overall colour barely tells healthy from diseased

This is the most important result in the exploration, and it is the one that shaped the modelling most.

The obvious first idea for a plant-disease model is that sick leaves are less green. To test that, we measured the *green fraction* — green divided by the total of all three channels, which removes the effect of overall brightness — and compared healthy against diseased leaves. We restricted the comparison to crops that have both conditions, so that the crop itself could not confuse the picture.

The two distributions sit almost exactly on top of one another. The median green fraction is **0.358 for healthy leaves and 0.354 for diseased ones**. A one-way ANOVA finds that difference statistically significant ($F \approx 94.8$, $p \approx 2.8 \times 10^{-22}$), but the effect size tells the real story: eta-squared is just **0.011**, meaning healthy-or-diseased status explains only about **1% of the variation** in greenness.

The difference is real and unimportant at the same time. It is significant only because the test runs on around 8,150 leaves, and with enough data almost any difference becomes significant. The reason behind it is simple once seen: **disease is local**. It is a few spots or lesions on an otherwise green leaf, and averaging over the whole leaf washes the symptom out.

This is the empirical case for using a convolutional network rather than hand-crafted whole-image features, and we were able to make that case before training a single model.

Finding 4 — the hand-crafted features are largely one feature

A correlation heatmap of the five numeric features that actually vary — file size, brightness and the three colour averages — shows brightness and the three colour channels almost perfectly correlated, with file size standing apart. Brightness tracks each colour channel at $r = 0.91\text{--}0.95$ and the channels track one another at $0.76\text{--}0.88$, while file size correlates only $0.14\text{--}0.26$ with the rest.

In other words, the four colour numbers are really one underlying "lightness" measurement recorded four times. They carry far less independent information than their count suggests, which reinforces Finding 3: whole-image colour holds very little that separates these classes.

Finding 5 — seen directly, the signal is obviously spatial

The last check was simply to look. Placing a healthy leaf beside a diseased one for four crops — Tomato, Apple, Grape and Potato — shows healthy leaves as an even green, and diseased leaves carrying localised damage: spots, lesions, ringed patches, yellowing, always confined to part of the leaf. The disease signal is plainly local and spatial, exactly as the statistics implied.

The same picture shows something else, and it is the thing that shadows the entire project: **every leaf sits detached, flat, and centred on a plain, uniform laboratory background**. This is a qualitative observation with no statistical test attached, and it did not need one.

The four difficulties this leaves us with

The exploration ends with four concrete problems that the rest of the project had to deal with. They are set out here as a list because Section 3 answers them one by one, in the same order.

- **Difficulty 1 — Severe class imbalance.** At roughly 36 to 1, the largest class has about thirty-six times as many images as the smallest. Left alone, a model will favour the common classes and overall accuracy will make it look better than it is.
- **Difficulty 2 — Single-condition crops and crop-disease entanglement.** Five crops appear under only one condition, so a per-crop healthy-or-diseased question is impossible to answer for them, and crop is statistically entangled with disease status.
- **Difficulty 3 — A clean laboratory background.** Every image is a detached leaf on a uniform studio backdrop. A model can take that shortcut instead of learning the disease, which inflates its scores and leaves a gap between this dataset and real field photographs.
- **Difficulty 4 — Tomato dominance.** Tomato alone accounts for roughly a third of all images, so the behaviour of the dataset as a whole is heavily influenced by one crop.

3. How the four difficulties were answered

The pre-processing step turned the raw images into something a model could train on. It is worth being clear about what that did and did not involve. The audit showed the data needs no classical cleaning: there are no missing values to fill, the geometry is already constant so nothing needs reshaping, and the images are already three-channel RGB. Pre-processing here was therefore not about repairing the data. It was about organising it so that no later number could be quietly corrupted, and about answering the four difficulties above.

The table below maps each difficulty to its answer. The rest of this section explains each one.

#	The difficulty	How it was answered
1	Severe class imbalance, 36 : 1	Class weights applied during training (0.26 for the largest class up to 9.44 for the smallest), plus per-class metrics instead of accuracy, so rare diseases cannot be averaged away.
2	Single-condition crops and crop-disease entanglement	The task was framed as flat 38-class prediction of the whole crop___condition label. A flat label space simply has no class for the combinations that do not exist.
3	A clean laboratory background	Light augmentation during training, and — later — a direct measurement of how much of the model's attention actually falls on the leaf, using the segmented masks held in reserve.
4	Tomato dominance	Covered by the same two measures as Difficulty 1: class weights during training, and macro-F1 with per-class recall for judging results.

Table 2. The four difficulties from the exploration, and the decision that answers each one.

Answering Difficulty 1 — the class imbalance

Two things were done about the imbalance, and they work at different points.

The first is **class weighting during training**. The loss the model is trying to reduce is weighted in inverse proportion to how common each class is, using weights calculated from the training portion only. They range from **0.26 for the largest class to 9.44 for the smallest**. In plain terms, a mistake on a rare disease costs the model proportionally more than a mistake on a common one, so it cannot simply ignore the small classes.

We chose class weighting over the alternative, which is physically oversampling the minority classes by making extra copies. Weighting runs on the GPU during training and adds no images to the dataset, whereas oversampling with a pre-processing loop inflates the dataset and runs on the CPU. Because the correction happens at training time rather than in the pipeline, it was written down explicitly as an obligation transferred to the modelling step — the one pre-processing decision that could be silently forgotten, and therefore the one worth stating twice.

The second measure is the **choice of measurement**, which Section 4 covers in full. In short, we judge models on macro-F1 and per-class recall rather than accuracy, so the imbalance cannot hide behind a flattering average.

Answering Difficulty 2 — the entangled labels

The answer here is a framing decision rather than a piece of code. Instead of asking the model two questions — which crop is this, and is it healthy? — we ask it one: which of the 38 crop-and-condition labels is this?

That single change dissolves the problem. Because the label space is flat, it contains only the combinations that actually occur in the data. There is no "healthy Orange" class for the model to be wrong about, because there is no such thing in the dataset. The impossible questions simply never get asked. A two-stage design that predicts crop first and disease second remains a possible alternative, but it would have to handle those cases explicitly, and nothing in the results suggested it was worth the extra complication.

Answering Difficulty 3 — the laboratory background

This is the difficulty we could reduce but not remove, and the report is careful about the difference.

During pre-processing the answer was **light augmentation**: random horizontal flip, small rotation and small zoom, applied to training images only. Like normalisation, augmentation is defined as a layer inside the model rather than as a change to the stored data. Keras augmentation layers are active during training only, so augmentation can never leak into the validation or test data. It adds variety within each class and acts as a mild regulariser. It is honest to say that its effect on the laboratory-to-field gap is modest.

The more substantial answer came later, in the modelling work, and it was to stop assuming and start measuring. The segmented version of the dataset — held in reserve since the exploration precisely for this — gave us a free outline of every leaf. That let us measure how much of the model's attention actually falls on the plant rather than the backdrop. Section 9 reports what that measurement found, including the part that was uncomfortable.

Answering Difficulty 4 — Tomato dominance

Tomato dominance is really the class imbalance seen from a different angle, and the same two measures cover it. Class weights stop the ten Tomato classes from dominating the training signal, and macro-F1 stops them from dominating the score, since every class contributes equally to that average regardless of how many images it has. It is worth noting that Tomato classes still account for several of the largest confusion pairs at the end of the project, which Section 8 discusses — the difficulty was managed, not made to disappear.

The other pipeline decisions

Four further decisions were made in pre-processing. They do not answer one of the four difficulties directly, but each protects the integrity of everything measured afterwards.

- **A stratified, leakage-free split, built at file level.** This is the single most consequential step in the whole pipeline, and Section 4 explains why it matters so much. Every image path and label was listed, and a stratified split was applied twice — first to separate the test set, then to take validation out of what remained — producing **70 / 15 / 15**: 38,013 training images, 8,146 validation and 8,146 test. Because the operation works on lists of files, the three sets cannot overlap. We verified this programmatically: all 38 classes are present in each of the three sets, and the sets are asserted to be mutually disjoint.
- **Why not the easy shortcut.** A tempting alternative is to shuffle and batch the dataset and then split it by taking and skipping batches. We rejected that explicitly. It is not stratified, so with 38 classes at 36 : 1 the rare classes can end up under-represented or missing from validation altogether. And it is not guaranteed leakage-free, because the underlying stream reshuffles on each pass, so the skipped portion is not the complement of the taken one and the same image can surface in two splits. Either problem would quietly corrupt every number downstream.
- **Resizing to 128 × 128, recorded as a compute trade-off rather than a necessity.** The images are already uniform, so no resizing is needed for consistency. We downsampled purely to reduce memory use and speed up training. This is a conscious deviation from our own finding that the disease signal is local — reducing resolution throws away some of the very detail that carries it. It was recorded as such and flagged for revisiting, and Section 8 shows exactly which class ended up paying for it.
- **Normalisation as a model layer, not a change to the data.** Pixel values are scaled from 0–255 to 0–1 by a Rescaling layer placed first in the network, rather than being rescaled in the stored dataset. This guarantees that training, validation and inference all use exactly the same scaling, and it means the eventual Streamlit application can hand raw images straight to the model.

One thing we deliberately did **not** do is classical feature engineering or dimension reduction. The exploration had already shown that hand-crafted whole-image features are highly redundant and barely separate the classes, and reducing the pixel space with something like PCA would discard precisely the local structure the model needs. In an image pipeline of this kind the feature engineering is the preparation of the raw pixels — the resizing, normalisation and augmentation above — rather than the construction of tabular features.

4. What "good" means here

What kind of problem this is

The task is supervised, single-label, multi-class classification. The input is a 128 × 128 colour photograph of one leaf; the output is exactly one of 38 mutually exclusive labels. There is no ordering among the labels and no continuous quantity to predict, so this is not regression. Every training image arrives with a ground-truth label supplied by the dataset's own folder structure, so it is not clustering either.

Why macro-F1 is the measure

Every comparison in this report is made on **macro-F1**. That means the F1 score is calculated separately for each of the 38 classes and then averaged without weighting, so each class contributes equally no matter how many images it has.

The alternative, plain accuracy, weights every image equally — which on this dataset means weighting the common classes heavily and the rare ones barely at all. The two dummy models turn that from an argument into a demonstration. A model that does nothing but predict the largest class every time achieves **10.1% accuracy**. That is a respectable-looking number, and it describes a model with no ability whatsoever. Its macro-F1 is **0.005**.

For a diagnostic tool the rare disease is precisely the one a grower most needs identified, so the measure has to be one that refuses to average it away.

What else we report

Macro-F1 is the decision metric, but one number cannot say much about *how* a model fails, so three other views are used throughout.

- **Accuracy** appears alongside macro-F1 for continuity with the published work on this dataset, and because the gap between the two is informative in itself. A wide gap means the model is doing well on the common classes and badly on the rare ones.
- **Per-class recall** is what an average conceals. It names the disease being missed, rather than reporting that some disease is being missed. The lowest per-class recall is quoted for the final model as a worst-case guarantee.
- **The confusion matrix** records not only how often the model is wrong but what it mistakes for what. That distinction matters: confusing two diseases of the same crop is a very different failure from confusing a diseased leaf with a healthy one, and Section 8 rests on it.

The four conditions that make eleven models comparable

Eleven numbers only form a comparison if they were produced under the same conditions. Four things were fixed before any model was trained, and none of them changed afterwards.

- **One data split, shared by everyone.** The 70/15/15 split described in Section 3 was distributed to the team as CSV files rather than regenerated locally, and every notebook prints the split's fingerprint at startup and halts if it does not match. That guard was not decorative: it caught an early run that had quietly built a split of its own. That run was discarded and repeated.
- **One metric.** Macro-F1 on the validation set, computed the same way for every model.
- **One approach to the imbalance.** Balanced class weights everywhere, with light augmentation additionally on the convolutional models. The classical models work from pre-computed features and are not augmented.
- **A sealed test set.** The 15% test partition was opened once, at the very end, for one model. Every other comparison and decision in this report was made on validation. Over-fitting during development was judged from the gap between training and validation curves, not by spending the test set early.

One further point governs how the results table should be read. All training ran on cloud GPUs, and repeating an identical run in a different session produces a slightly different result. We measured that variation at roughly **±0.02 macro-F1**. Differences between models smaller than that should not be treated as real, which is why Section 6 is careful to argue the final decision on more than a narrow numerical margin.

Taken together these four decisions cost very little to make and are close to impossible to retrofit. A comparison assembled from models trained on different splits, scored on different metrics, or tuned against a test set that was consulted along the way is not a comparison at all, however carefully the numbers are tabulated afterwards.

5. The ladder of models

The eleven models were not tried at random. Each one was built to answer a specific question, and the answer determined what got built next. This section follows that chain of questions in order. **Appendix A** lists all eleven models with their scores in a single table.

Read as a whole, the ladder tells a story in three movements. The first four rungs establish how much of the problem can be solved without looking at *where* anything is in the image: not much. The fifth shows how far careful feature engineering can push that, and it is further than one might expect. From the sixth rung onwards the models

learn their own features from raw pixels, and the numbers change character — the question stops being whether the problem can be solved and becomes which of several good solutions to prefer.

Question 1: Where does the floor sit, and is accuracy safe to use?

Before judging any real model, we need to know what a score even means on this data. Two dummy models establish that. Neither learns anything at all: one always predicts the most common class, and the other draws predictions at random in proportion to how often each class appears.

The answer: the floor sits at **0.005 and 0.026 macro-F1**. Those are the numbers a model must clear to be said to have learned anything whatsoever. And no, accuracy is not safe to use — the first dummy scores 10.1% accuracy against 0.005 macro-F1, and that gap is the clearest possible warning against reading accuracy on this dataset.

Question 2: Are the whole-image features linearly separable?

The exploration produced six whole-image summary features — file size, brightness and the three colour averages, plus derived measures. The obvious first question is whether a straight line through that six-dimensional space can separate the 38 classes. A logistic regression answers it.

The answer: barely. **0.274 macro-F1**. Better than nothing, and far above the floor, but nowhere near useful. That is roughly what Finding 3 in the exploration predicted would happen.

Question 3: What is non-linearity alone worth?

The next rung changes exactly one thing. Same six features, same split, same weighting — but a Random Forest instead of a linear model, so the model can now capture non-linear relationships. Anything the score gains is attributable to the algorithm and nothing else.

The answer: about **+0.11 macro-F1**, from 0.274 to 0.382. Real, but modest. Hold that number, because the next question makes it meaningful.

Question 4: How far can hand-built features go?

Now we keep the Random Forest and change the input instead, expanding from six features to about 160. The richer feature set is worth describing, because it is what the deep models had to beat. It consists mainly of colour histograms computed in two colour spaces — the familiar red, green and blue channels, and the hue, saturation and value representation that separates colour from brightness — together with texture descriptors derived from a grey-level co-occurrence matrix, which measures how often particular pairs of intensities appear next to one another. The six whole-image summaries are kept alongside them. Nothing here is exotic. It is the standard toolkit a practitioner would reach for before turning to deep learning.

The answer: **0.906 macro-F1** — a gain of about **+0.52** over the same algorithm on six features. This is the baseline, and it is a genuinely strong result for a classical model.

Putting Questions 3 and 4 side by side gives the most transferable lesson of the classical tier: **the features are worth about five times what the choice of algorithm is worth**. It also changes how the baseline's 0.906 should be read. The credit belongs to the inputs rather than to the ensemble — the colour histograms alone account for around **83%** of the model's feature importance, against roughly **7%** for the six whole-image summaries.

A Random Forest was chosen for these rungs because it is a bagging ensemble: many decision trees, each grown on a resampled subset of the data and a random subset of the features, with their votes averaged. That construction suits the job. It copes with features on wildly different scales without rescaling them, it accepts class weights directly, and it reports which features it relied on — which is what made the analysis above possible in the first place. A boosting model, which builds trees sequentially so that each corrects the last, was not built. It would very likely have improved on 0.906 by a few points, but not by the 0.067 that separates the classical tier from the convolutional one, for the reason set out next.

Why the classical tier stops where it does

A colour histogram counts how many pixels of each colour an image contains. That is enough to notice that a leaf has off-colour pixels, which is often enough to know that something is wrong with it. What a histogram cannot do is say **where** those pixels are — whether they form a ring, a streak along a vein, or a scattering of small spots. Two diseases that discolour the same amount of tissue in different patterns produce nearly the same histogram.

The ceiling at 0.906 is therefore a property of the representation, not of the model sitting on top of it. Histograms are spatially blind, and disease identity is spatial. No number of additional trees, and no switch from bagging to boosting, can supply information that the inputs do not contain. Closing that gap needs a model that builds its own inputs.

One caution belongs in the record. Among the six whole-image summaries, **file size in kilobytes** ranked as one of the most influential, at 0.20 importance in the six-feature Random Forest. File size is an artefact of JPEG compression, not a fact about plant biology. It is the earliest sign in this project that a model can score well by reading properties of the photograph rather than properties of the plant — a concern that returns, measured directly, in Section 9.

Question 5: Does learning features beat designing them?

A convolutional network does not receive a summary of the image. It receives the pixels, and it learns for itself which small local patterns are worth detecting — an edge, a boundary between healthy and dead tissue, the concentric banding of a particular lesion — and then which arrangements of those patterns indicate which disease. The spatial information that a histogram throws away is exactly what this architecture is built to keep.

The answer: yes, decisively. 0.973 macro-F1 from four convolutional blocks trained from a random start at 128 pixels, with **263,654 parameters**. That is a jump of 0.067 over the strongest classical model, achieved with fewer stored numbers than the classical pipeline extracts features per image.

Two properties of the architecture do the work. The first is **locality**: each filter looks at a small patch at a time, which matches the fact that a lesion is a local event on an otherwise ordinary leaf. The second is that **the same filter is applied everywhere** in the image, so a pattern learned from a lesion near the leaf tip is recognised when it appears near the base. Together these let the network learn a small vocabulary of lesion appearances and then detect them anywhere — exactly the behaviour the problem calls for, and exactly what a global histogram discards.

The prediction made in the exploration — that the disease signal is local and spatial, and that a model would have to see structure rather than summaries — is paid out in full by this single result.

Question 6: Was there headroom left in the settings?

Having got a strong result from an architecture chosen by hand, the obvious worry is whether that configuration was lucky rather than good, and whether a systematic search would find something better. That is the entire purpose of the tuned CNN: not to squeeze out an extra fraction of a point, but to find out whether headroom was being left on the table.

A Bayesian search using keras-tuner ran fifteen trials over five hyper-parameters — learning rate, dropout, the width of the dense head, the width of the convolutional base, and the strength of augmentation. Bayesian optimisation was used rather than an exhaustive grid because it builds a model of which regions of the search space look promising and concentrates its trials there, which matters when every trial costs GPU time. To keep the search affordable, trials were ranked on a stratified subsample of the training data and the winning configuration was then retrained in full.

The answer: no headroom. The search rediscovered the hand-chosen configuration on four of the five parameters, and the retrained winner scored **0.970** against the original 0.973 — a difference well inside run-to-run variation.

This is a useful negative result rather than a wasted experiment. It says the remaining errors at this level are not caused by badly set dials, and therefore that further progress had to come from a different architecture rather than from better tuning of this one. That conclusion is what sent the project to transfer learning.

Question 7: Do pretrained features help on this data at all?

Transfer learning starts a model from weights already trained on a different, much larger dataset — here ImageNet, roughly 1.4 million photographs of everyday objects and scenes. The idea is that the early layers of any vision model learn generic things: edges, corners, textures, colour gradients. Those are as useful for a leaf lesion as for a lamppost, so rather than learning them again from scratch, the model inherits them and only has to learn what is specific to the task.

Four backbones were fine-tuned on the shared split, chosen to cover the design space rather than to be individually likely winners. **MobileNetV2** is the lightweight option, designed for phones, and was the cheapest way to test whether pretrained features helped at all. **ResNet50** is the long-standing reference architecture against which most others are reported. **EfficientNetB0** represents a family designed by scaling depth, width and resolution together rather than one at a time. **DenseNet-121** reuses features aggressively, with every layer inside a block receiving the outputs of all those before it, which lets it be deep without being wide. Between them they cover four different points on the size-versus-capacity trade-off.

All four were trained under a single standardised protocol, which was itself the most consequential piece of tuning in the project. It has three parts.

- **Two phases.** The backbone is frozen first and only the new classification head is trained, at a relatively high learning rate. Once the head is sensible, the backbone is unfrozen and the whole network trains together at a learning rate two orders of magnitude lower, so that the inherited features are refined rather than overwritten.
- **Normalisation layers stay frozen.** The BatchNorm layers keep their inherited running statistics throughout. Letting them adapt on small effective batches destabilises fine-tuning, and freezing them was the single change that made these runs reliable.
- **Verification that fine-tuning actually happened.** Each run asserts that the number of trainable parameters really does rise when the backbone is unfrozen. Without that check, a configuration error can leave the second phase training nothing at all while still producing a plausible-looking score.

The answer: yes, but less than expected. MobileNetV2, the cheapest test, reached **0.919** — below the scratch CNN. ResNet50 reached **0.978** and EfficientNetB0 **0.982**, both above it. Three of the four cleared the scratch CNN, and **DenseNet-121 cleared it clearly at 0.9918**.

The tier as a whole delivered a smaller improvement than the literature on transfer learning might lead one to expect, and the reason is instructive: transfer learning helps most when training data is scarce, and 38,013 training images of a visually uniform kind is not scarce. A network trained from scratch has enough data here to learn good features on its own, so the inherited head start is worth less than it would be otherwise.

That observation looks like a limitation of transfer learning on this dataset. Section 6 argues that it is in fact the strongest reason to prefer the pretrained model, because the dataset this tool will eventually have to handle is not this one.

6. Choosing the final model

The decision rests on validation performance. Across all eleven models, DenseNet-121 has the highest macro-F1 on the shared validation set, at **0.9918 against 0.973** for the scratch CNN, its nearest architecturally distinct rival. That is the primary and simplest reason it was taken forward.

But the margin is 0.019, which is close to the ± 0.02 run-to-run variation documented in Section 4. It would be careless to rest the decision on the headline number alone, so what follows is what supports it.

It is much the better base for a field version

This is the main argument, and it follows directly from the observation that closed Section 5. Pretrained weights pay off most when training data is scarce. PlantVillage is not scarce, which is exactly why transfer learning delivered a smaller advantage here than it usually does. But the version of this tool that matters is the one that works on leaves photographed on living plants, in daylight, against soil and other foliage — and *that* data will be scarce. Field imagery arrives in the hundreds or low thousands of images, not the tens of thousands.

In that situation the two candidates are not comparable. DenseNet-121 carries 7.1 million parameters of general visual structure learned from 1.4 million varied natural photographs. Adapting it to a small field dataset is a shallow re-fine-tune of a network that already knows what daylight, occlusion and cluttered backgrounds look like. The scratch CNN carries 263,654 parameters learned exclusively from detached leaves on a plain grey backdrop; adapting it to field conditions is much closer to starting again, on a dataset far too small to start from.

Choosing DenseNet-121 therefore costs nothing in laboratory performance and keeps open the one route that leads to a deployable tool.

It is worth stating the counterfactual, because it clarifies what the argument does and does not claim. If the objective were only to post the best number on this benchmark and stop, the case would be much weaker: the margin over the scratch CNN is close to noise, the smaller model is twenty-seven times lighter, and simplicity would be a reasonable tie-breaker. The objective is not that. It is a tool that eventually works on a photograph taken in a field, and under that objective the two candidates are not close.

The errors it removes are the ones that were hardest to remove

Against the scratch CNN, the final model reduces validation errors by a factor of 3.4, from 190 to 56. What matters more than the count is *where* those errors were. The final model holds a minimum per-class recall of **0.9481**, with only one class of 38 below 0.95 and **20 of 38 recalled perfectly**. Because only a single class now sits below that threshold, every class the scratch CNN's error analysis had named as hard — Tomato Early blight, Target Spot, Corn Northern Leaf Blight, Apple scab — has been lifted clear of it.

A model that has merely got lucky improves where there was already room to improve. Repairing the tail is what macro-F1 was chosen to reward in the first place, and it is what a diagnostic tool needs, because the disease a grower most needs identified is usually not the common one.

What is left over is largely not fixable by choosing a different model

Section 8 shows the 57 remaining test errors resolving into a small number of recurring pairs, most of which are either genuine biological ambiguity or low-cost confusions between two hosts of the same pathogen. The error budget on this dataset is close to its irreducible part, which is a further reason to stop optimising here and change the data instead.

What did not decide it

It is worth saying what was considered and set aside. The interpretability work described in Section 9 measures where each model looks, and it does not separate the two candidates in a way that would justify overriding their scores. Both attend preferentially to the leaf, and the small difference between them runs in the pretrained model's *disfavour* rather than its favour. That measurement is reported in full because it is informative about the models, but it plays no part in the selection, and this report does not use it to argue for the model that was chosen.

What the choice costs

The final model has 7,076,454 parameters against the scratch CNN's 263,654, and takes 72 minutes to train on a T4 GPU against a few minutes. The argument from simplicity is a real argument and was taken seriously; it loses here on merit. Inference cost is not a differentiator — both models are fast enough for interactive use and small enough to serve inside a Streamlit application — and the training cost is paid once.

7. The final model, and the sealed test set

What the final model is

DenseNet-121 is a convolutional network whose distinguishing feature is **dense connectivity**: within each block, every layer receives the outputs of all the layers before it. That reuse lets the network be deep without being wide, which is why it reaches this depth at a little over seven million parameters.

It was initialised with ImageNet weights and trained on the shared split at 128 pixels, with a batch size of 32 and balanced class weights, following the two-phase protocol from Section 5. The first phase trained the classification head alone, with the backbone frozen, at a learning rate of 1e-3 for five epochs. The second unfroze **306 of the network's 427 layers** — all 121 BatchNorm layers stayed frozen — and continued at 1e-5 with early stopping, which triggered at epoch 19 with the best weights taken from epoch 15. Trainable parameters rose from **38,950 to 6,909,158** between the phases, confirming that fine-tuning actually engaged, and balanced class weights were verified as passed in both phases. Total training time was **72 minutes on a T4**.

The sealed evaluation

The test partition had not been read by any model before this point. It was opened once, for this model, and has not been opened since. The evaluated checkpoint is identified by the MD5 hash afe3478450c41d91a6ac1e57a2ada44d, so the result is tied to a specific file rather than to a description of one.

DenseNet-121	Validation (8,146)	Test (8,146, sealed)
Macro-F1	0.9918	0.9919 95% CI [0.9896, 0.9941]
Accuracy	0.9931	0.9930
Errors	56 of 8,146	57 of 8,146 (0.70%)
Minimum per-class recall	0.9481	0.9481

Table 3. The final model on the validation set and on the sealed test set. The confidence interval is a bootstrap 95% interval over the test predictions.

The confidence interval is worth a word, because a single number computed once invites more confidence than it deserves. It was obtained by bootstrapping: the test predictions are resampled with replacement many times, macro-F1 is recomputed for each resample, and the interval covers the middle 95% of those values. It describes how much the score would be expected to move if a different sample of 8,146 leaves of the same kind had been drawn. At ± 0.0023 it is narrow, which is a consequence of the test set being large.

The gap between validation and test is **+0.0001** — the model scores fractionally better on data it has never been evaluated against than on the data used to choose its checkpoint. There is no measurable over-fitting. This is precisely the result that the sealed protocol exists to make believable: had the test set been consulted during development, a small gap like this would carry very little information, because the test set would by then have become part of the development loop.

Set against the ladder, the final model reduces the error rate of the strongest classical model by roughly a factor of eleven, and that of the scratch CNN by 3.4. In absolute terms it misidentifies **57 leaves out of 8,146 — about one in 143**.

8. Where the remaining errors are

Fifty-seven errors is a small enough number to examine individually, and doing so is more informative than the headline score. They are not scattered evenly across the problem. They fall into **31 distinct pairs** of true and predicted class, and the ten largest pairs account for **63% of all errors**. The five largest are below.

True class → predicted class	Images	Share of class	Share of all errors
Tomato Spider mites → Tomato Target Spot	9	3.6%	15.8%
Tomato Yellow Leaf Curl Virus → Tomato Bacterial spot	8	1.0%	14.0%
Tomato Late blight → Potato Late blight	5	1.8%	8.8%
Tomato Late blight → Tomato Early blight	4	1.4%	7.0%
Corn Cercospora Gray leaf spot → Corn Northern Leaf Blight	4	5.2%	7.0%

Table 4. The five largest confusion pairs on the sealed test set. Share of class is the pair's size relative to that true class's test images; share of all errors is relative to all 57.

Three patterns run through them, and each means something different for how the tool would behave in use.

- **Confusion between similar diseases of the same crop.** Spider mite damage read as Target Spot; Late blight read as Early blight. These are the fine-grained core of the task — distinctions a non-specialist would also struggle to make from a photograph — and they are the errors one would most expect to survive at this level of accuracy. They are also the errors with real diagnostic cost, since the treatment differs.
- **Confusion between crops sharing one pathogen.** Tomato Late blight predicted as Potato Late blight, five images, with one error in the reverse direction. Tomato and potato are both Solanaceae, and *Phytophthora infestans* produces genuinely similar lesions on both. Judged as a diagnosis rather than as a label, this is the least costly error in the table: the pathogen is identified correctly and only the host is wrong, and the host is something the person holding the phone already knows.
- **One stable ceiling.** Corn Cercospora leaf spot mistaken for Corn Northern Leaf Blight — four images, 5.2% of that class. This is the class that sets the minimum per-class recall at 0.9481, and that recall is 0.9481 on validation *and* on test, which makes it a stable property of the data rather than sampling noise. The same confusion appeared in every model built in this project, from the Random Forest upward. At 128 pixels the two lesion types are close to indistinguishable, and no amount of further training will separate them at that input resolution. This is where the resolution trade-off taken in Section 3 finally shows its price.

Error analysis was more productive earlier in the project than it is at this level. When the scratch CNN was the strongest model, per-class recall named a hard set of four classes, and those turned out to be exactly where the pretrained backbones gained their advantage. At 0.9919 the remaining signal is thinner, and what the analysis now supplies is two specific, testable proposals rather than a general direction. The Corn pair is a resolution hypothesis, testable by retraining that crop's classes at 256 pixels. The Tomato Spider mites pair — the single largest, at 15.8% of all errors — is a candidate for augmentation targeted at that class rather than more epochs applied to all of them.

It is also worth noting what does **not** appear in the table. None of the five largest pairs involves a diseased leaf being classified as healthy, which is the failure mode with the highest cost in practice — a grower told that a sick plant is fine will act on that, or rather will not act at all. That such errors are absent from the top of the list is a meaningful property of the model, and it is a direct consequence of choosing a metric that would not let the rare and difficult classes be averaged away during development.

9. Looking inside the model

A model that is right 99.3% of the time is still a model that gives an answer without a reason. For a diagnostic tool that is not quite good enough: a grower shown a disease name has no way to tell a confident, well-founded prediction from a lucky one. This section describes what can be seen inside the final model when it makes a decision, and what that reveals about both its strengths and its limits.

What Grad-CAM shows

Grad-CAM recovers something close to a reason. It reads the last convolutional layer of the network — the deepest point at which the information still has a location in the image — and measures how strongly each region there pushed the prediction towards the class the model actually chose. The output is a heat map laid over the original photograph: warm where the evidence for the decision came from, cool where it did not.

It answers one question well — *where did the model look?* — and should not be stretched further. It does not say what would have changed the model's mind, and it does not show that the model understands anything about plant pathology. It is a diagnostic instrument, not a proof.

Turning heat maps into a number

Heat maps are persuasive and easy to over-read, so we converted the question into a measurement. This is where the segmented version of the dataset, held in reserve since the exploration, finally earned its keep. It supplies a free outline of every leaf, produced by the dataset's own authors.

For each image, the share of the heat map falling inside that outline was divided by the share of the frame the leaf occupies. The resulting figure, called **concentration**, reads simply:

- **1.0** means no preference at all — the model's attention is spread as though the leaf were not there.
- **Above 1.0** means the model concentrates on the leaf.
- **Below 1.0** means it preferentially reads a background that contains no disease information whatsoever.

The measurement covers the **7,967 validation images** that have a segmented counterpart, 97.8% of the validation set. Leaf area averages 47.4% of the frame, identically for both models, since that is a property of the photographs rather than of the networks.

Two things make this a fair probe. The masks were produced by the dataset's authors rather than by us, so they are not tuned to flatter either model. And neither network ever saw them — all training used the unsegmented colour images. The measurement is therefore genuinely external to the models being measured, which is what makes the comparison between them meaningful rather than circular.

Model	Attention on leaf	Concentration (native)	Concentration (matched 4×4)
Scratch CNN	56.7%	1.226	1.169
DenseNet-121	53.3%	1.148	1.134
Difference	-3.4 pp	-0.078	-0.034

Table 5. On-leaf attention across 7,967 masked validation images. Concentration is attention share divided by leaf area. The matched column recomputes both models' heat maps on a common 4×4 grid; the 95% intervals on the difference are $[-0.090, -0.066]$ native and $[-0.043, -0.025]$ matched.

What it found

Both models prefer the leaf. The scratch CNN places 56.7% of its attention on a leaf occupying 47.4% of the frame, and the final model places 53.3% there. Both sit comfortably above 1.0 concentration, so neither is reading the backdrop in preference to the plant. Both also leave a substantial share of their attention on a plain grey background that carries no disease information at all, which is a useful corrective to scores in the high nineties.

The final model's concentration is slightly the lower of the two, and the difference is statistically real. This is uncomfortable and is reported because it is true. Part of it is an artefact of measurement: DenseNet-121's last convolutional layer produces a 4×4 grid at this input size, where the scratch CNN's produces 16×16 , and a coarser map cannot help but spill across a leaf edge. Recomputing both on a common 4×4 grid controls for that, and the difference more than halves but does not disappear.

What this means practically is that the final model gives **no evidence** of being less dependent on the laboratory backdrop than the from-scratch network, and no claim of that kind is made for it anywhere in this report. Whether it would hold up in a field is an open question that only field images can answer.

The useful finding: attention tracks correctness

Splitting concentration by outcome produces the most practically valuable result of the interpretability work. On the images the final model classifies **correctly**, concentration averages **1.135**. On the images it gets **wrong**, it averages **1.009**. The difference is +0.126, with a 95% interval of $[+0.053, +0.203]$ and a Mann-Whitney p-value of 7.1×10^{-4} . The scratch CNN shows the same effect more weakly: 1.171 against 1.080, a difference of +0.091.

A concentration of 1.009 is no leaf preference at all. On the images the final model gets wrong, its attention is spread as though the leaf were not there. Misclassification here is not a subtle judgement between two plausible diseases — it coincides, measurably, with the model reading the wrong part of the frame.

That is directly usable. Attention concentration can be computed at inference time and used as a confidence signal, flagging predictions whose heat map sits off the leaf for human review, without needing a ground-truth label to check them against. The effect is stronger in the final model than in the scratch CNN, and it is a concrete route to a safer diagnostic tool rather than merely a higher score.

10. Difficulties encountered along the way

The account above reads more smoothly than the work did. Four things made it harder than the section order suggests, and each is worth recording because each would recur for anyone attempting the same thing.

Telling a real signal from an important one

The main scientific obstacle was the gap between a result that is statistically real and one that actually matters. The exploration's central finding — significant at $p \approx 10^{-22}$, negligible at an effect size of 0.011 — is exactly the kind of result that is easy to report the wrong way round. Reading it correctly is what ruled out an entire class of approaches before any of them was built.

The same difficulty returned at the other end of the project in a different form. At 0.9918 against 0.973, with documented run-to-run variation of ± 0.02 , the headline margin between the two finalists was not by itself evidence, and the selection had to be argued on error structure and on the field-adaptation case instead.

The data itself was the binding constraint

Acquisition was straightforward; the dataset's character was not. PlantVillage is detached leaves photographed flat under even light, and no field imagery was available to the project at any point. The laboratory-to-field question could therefore be reasoned about, and — through the attention measurement — partly probed, but never actually measured against real field photographs.

Two failure modes that fail silently

Both cost real time, and both are worth naming because neither announces itself.

The first is a fine-tune whose second phase leaves the backbone frozen. Training completes, the loss curve looks plausible, and the score is simply lower than it should be, with no error message anywhere. It is caught only by asserting that the trainable-parameter count rises between phases — and that assertion was added after the problem had already affected earlier runs.

The second is a notebook that regenerates its own split instead of loading the shared one. It produces numbers that look comparable and are not. The split fingerprint printed at every startup was added for exactly this reason, and duly caught one such run.

Compute, and what it cost

All training ran on cloud GPUs with session limits and recycled runtimes. That is the source of the ± 0.02 variation quoted throughout. It also cost the project a small number of early checkpoints and notebooks when runtimes were recycled, which is why the final model's checkpoint is identified by hash rather than by description. Where a figure comes from a surviving artefact and an earlier document records a different one, the surviving artefact is the figure of record. This affects no ordering in the results table and no conclusion in Section 6.

11. What this report does not establish

The following are stated plainly so that a reader does not have to infer them.

- **Every number here is a laboratory number.** PlantVillage is detached leaves photographed flat against a plain background under even light. A deployed system sees leaves on living plants, in variable daylight, against soil and other foliage, often partly occluded and rarely centred. The published literature on this dataset reports large performance drops under those conditions, and nothing in this report contradicts that expectation.
- **No field-robustness claim is made for the final model.** The attention measurement in Section 9 gives no evidence that the pretrained model depends less on the laboratory backdrop than the from-scratch one. The argument in Section 6 is about which model is the better *starting point* for a field version, not about which performs better in a field today. That distinction matters and is not being blurred.

- **The attention comparison is partly confounded by resolution.** Matching both models on a 4×4 grid controls for it and the effect survives, but it more than halves. A cleaner test would compare heat maps drawn from an intermediate DenseNet block at 8×8 or 16×16 .
- **No boosting model was built.** Section 5 gives the reasoning — the classical ceiling is set by the features rather than by the ensemble — but the comparison between bagging and boosting on this feature matrix remains a gap.
- **Run-to-run variation is roughly ± 0.02 macro-F1.** Differences smaller than that between any two rows of the appendix table should not be read as meaningful. This is why the argument in Section 6 rests on error structure and on the field-adaptation case rather than on the headline margin.
- **The transfer models were not compared at their intended input sizes.** All four backbones were fine-tuned at 128 pixels, which is a fair comparison at fixed compute but not a fair comparison of the architectures as their authors designed them.
- **The 128-versus-256-pixel resolution comparison was never closed.** It was flagged in the pre-processing as something to check against validation macro-F1, and Section 8 shows precisely which class is paying for it, but the experiment itself was not run.

12. Outlook

Further gains on this dataset are close to meaningless. At 0.9919 with twenty of thirty-eight classes recalled perfectly, the remaining errors are either genuine biological ambiguity, as in the Corn pair, or confusions whose practical cost is negligible, as in the Tomato-to-Potato Late blight pair. The task in front of this model is not a higher number. It is a different dataset, and everything that would genuinely move the project forward sits on the far side of that.

One qualification belongs at the head of this list

This is a capstone project with a fixed timetable and no budget for data collection, so none of what follows is work this team is in a position to carry out. Acquiring field imagery means going to farms, or licensing photographs, or building a collection effort with agronomists to verify the labels — months of work and a budget, neither of which exists here. The same applies, in smaller measure, to the GPU hours the remaining experiments would consume; two experiments planned during pre-processing were already dropped for want of that budget.

What follows is therefore not a plan being executed. It is set down because the limitations above point at specific, answerable questions rather than vague ones, and because anyone taking this model further should not have to rediscover them. If the resources existed and this tool had to work in a field rather than on a laboratory bench, this is the order in which we would tackle it.

- **Acquire or borrow field imagery** — leaves in situ, phone cameras, mixed backgrounds and lighting — and measure the drop rather than argue about it. This is by a wide margin the most informative thing left to do, and it is the step for which the final model was chosen.
- **Adapt the final model to that imagery with a shallow re-fine-tune.** Unfreezing 306 of 427 layers maximises fit to the laboratory distribution; a shallower unfreeze that preserves more of the inherited general structure is the natural configuration to test against field data once it exists.
- **Implement attention concentration as a runtime confidence signal**, flagging predictions whose heat map sits off the leaf for human review. The measurement already exists; what is needed is the plumbing and a threshold chosen against a cost model rather than against accuracy.
- **Retrain the Corn classes at 256 pixels** to test the resolution hypothesis behind the only class below 0.95 recall, and apply targeted augmentation to the Tomato Spider mites class rather than more epochs to all of them.
- **Build the gradient-boosting model on the existing feature matrix** and close the bagging-versus-boosting comparison. The features already exist, so this is a fit call rather than a pipeline.

- **Consider a pretraining corpus closer to the domain.** What ImageNet contributes here is low- and mid-level texture and edge structure rather than its semantic categories, which are of little use for flat leaf scans. Weights pretrained on in-situ plant photography would be the principled fix for the laboratory-to-field gap.

13. Conclusion

The objective set at the start of this project has been met. A fine-tuned DenseNet-121 identifies all 38 crop-and-condition classes from a single leaf photograph at **0.9919 macro-F1** on data it had never seen, with a validation-to-test gap of +0.0001 and a worst class still recalled at 0.9481. The comparison behind that choice was made on one split, with one metric, under one set of conditions, and the reasoning is recorded rather than assumed.

Three things that will outlast the score

The representation dominates the choice of model. The same Random Forest gains 0.52 macro-F1 from richer features and only 0.11 from non-linearity, and the entire classical tier is capped by the spatial blindness of colour histograms rather than by the capacity of the ensemble built on them. Convolution's advantage here is specific and nameable: it is the ability to say *where* a discoloured pixel sits, not merely that one exists.

Accuracy and attention are separable properties. The final model scores highest and attends to the leaf marginally less than a model scoring lower, so a project that selects on the score should not assume it has also bought better localisation. The useful corollary is that attention still tracks correctness within each model, and does so most sharply in the final one, which turns Grad-CAM from a presentation device into a runtime confidence signal.

A saturated benchmark is a warning rather than a success. At 0.9919 on sealed test data, with twenty of thirty-eight classes recalled perfectly, further gains on this dataset are close to meaningless. The remaining errors are either genuine biological ambiguity or confusions whose practical cost is negligible.

Where that leaves the project

Every number in this report is a laboratory number. PlantVillage is detached leaves photographed flat against a plain backdrop, and neither finalist has ever seen anything else. What the work delivers is a well-understood model, a documented comparison behind it, and a clear view of what would have to happen next.

What remains is not a better model on this dataset. It is a dataset that resembles the problem, and a model chosen so that it can be adapted to one. Everything from Section 4 onwards concerns how well a model can distinguish 38 conditions in photographs taken under controlled conditions, and the answer is: very well, with the residual difficulty concentrated in a handful of genuinely ambiguous pairs. Whether the same model can distinguish them on a leaf still attached to a plant, in ordinary daylight, against soil and other foliage, is a separate question that this dataset cannot answer and this project could not afford to ask.

The honest summary of the position is that **the modelling question has been answered and the deployment question has not yet been asked.** The value of the work done here is that it leaves that question in a good position to be asked.

Appendix A. The eleven models compared

All eleven models in the order they were built, scored on the shared validation set under the fixed conditions of Section 4. Macro-F1 is the decision metric throughout; accuracy is shown for context only. No test-set figure appears here — the sealed evaluation of the final model is reported in Section 7. Differences smaller than ± 0.02 macro-F1 fall inside documented run-to-run variation and should not be read as meaningful.

Model	What it is, and the question it answers	Macro-F1	Accuracy
Dummy — most frequent	Always predicts the largest class. Where does the floor sit, and is accuracy safe to use?	0.005	0.101
Dummy — stratified	Predicts by class frequency at random. A second no-skill reference point.	0.026	0.046
Logistic regression, 6 global features	Linear model on the whole-image summaries from the exploration. Are those features linearly separable?	0.274	0.320
Random Forest, 6 global features	The same six features under a non-linear ensemble. What is non-linearity alone worth?	0.382	0.507
Random Forest, ~160 features	Bagged trees on colour histograms and texture. How far can hand-built features go?	0.906	0.926
Scratch CNN	Four-block network learning from pixels at 128 px. Does learning features beat designing them?	0.973	0.977
Tuned CNN	The same architecture after a 15-trial Bayesian search. Was there headroom left in the settings?	0.970	0.973
Transfer — MobileNetV2	Lightweight ImageNet backbone, fine-tuned. Do pretrained features help on this data?	0.919	0.932
Transfer — ResNet50	Larger ImageNet backbone, fine-tuned on the canonical split.	0.978	0.981
Transfer — EfficientNetB0	Compound-scaled ImageNet backbone. Does a more efficient family do better?	0.982	0.984
DenseNet-121 — the final model	ImageNet backbone with dense connectivity, fine-tuned in two phases. The final model.	0.9918	0.9931

Table A.1. All eleven models on the shared validation set, in build order. Read from the top, the table tells the story in three movements: the first four rows establish how much of the problem can be solved without looking at where anything is in the image, the fifth shows how far careful feature engineering can push that, and from the sixth onwards the models learn their own features from raw pixels.

Code files

The work is contained in a sequence of notebooks. *Step1_PlantVillage_Exploration* builds the metadata table, the five figures and the fifteen-row data audit, which is also exported as *DATA_AUDIT_plantvillage.xlsx*. *plant_recognition_preprocess* builds the stratified, leakage-free split and turns the raw colour images into the three batched datasets, the normalisation and augmentation layers and the class weights. The modelling notebooks follow in build order — the two dummies and the classical models, the scratch CNN and its Bayesian tuning run, and one notebook per transfer backbone, ending with DenseNet-121. A separate interpretability notebook produces the Grad-CAM heat maps and the on-leaf attention measurement against the segmented masks, and a final notebook contains the single sealed test-set evaluation. Every notebook prints the split fingerprint at startup and halts on a mismatch.